Channel permutations × mixed-MXFP activations × HMT

Algorithm and implementation design for AutoRound, targeting Intel GPUs / XPU

Higher-fidelity 4-bit quantization

A verifiable route to XPU deployment

F1 Fixed K-axis permutations + MXFP4

F2 Mostly A4 + a small A8 subset

HMT Optional, paired, jointly optimized

13 pinned sources · Author figures and data · Concrete integration points · Staged development

One shared plan, two features, three execution tiers

Validate numerics and accuracy before choosing hardware paths. Keep all weights at 4 bits by default.

F1 / Regroup channels

Learn a fixed offline permutation

Permute activations and weights

Rebuild 32-element groups

Start with all-W4A4

F2 / Allocate precision

One gather, two partitions

Mostly MXFP4 activations

A small MXFP8 subset

MXFP4 weights in both branches

HMT / Optional transform

Rotate the low partition first

Start with fixed H32

Apply matched compensation

Defer learned rotations

Reference QDQ → Decode-fused XPU kernels → Capability-verified native ops

Evidence boundaries: verified facts vs. open questions

Author results, theoretical calculations, and development targets are different types of evidence.

Evidence	Inspected in this study	Does not establish
Code facts	Pinned AutoRound source; author algorithms and kernels	Production readiness of these paths
Author reports	Original figures, author CSV, pinned README results	Equivalent gains for the new features on XPU
Theory / design	Paired-transform algebra, budgets, alignment	Latency scaling with theoretical byte cost
Unverified	Candidate algorithms, gates, effort estimates	Achieved accuracy gains or speedups

Paper hosts were unreachable; evidence comes from author code, released figures, and original data.

Full revisions and hashes: evidence.json. This is neither a full-text nor an exhaustive literature review.

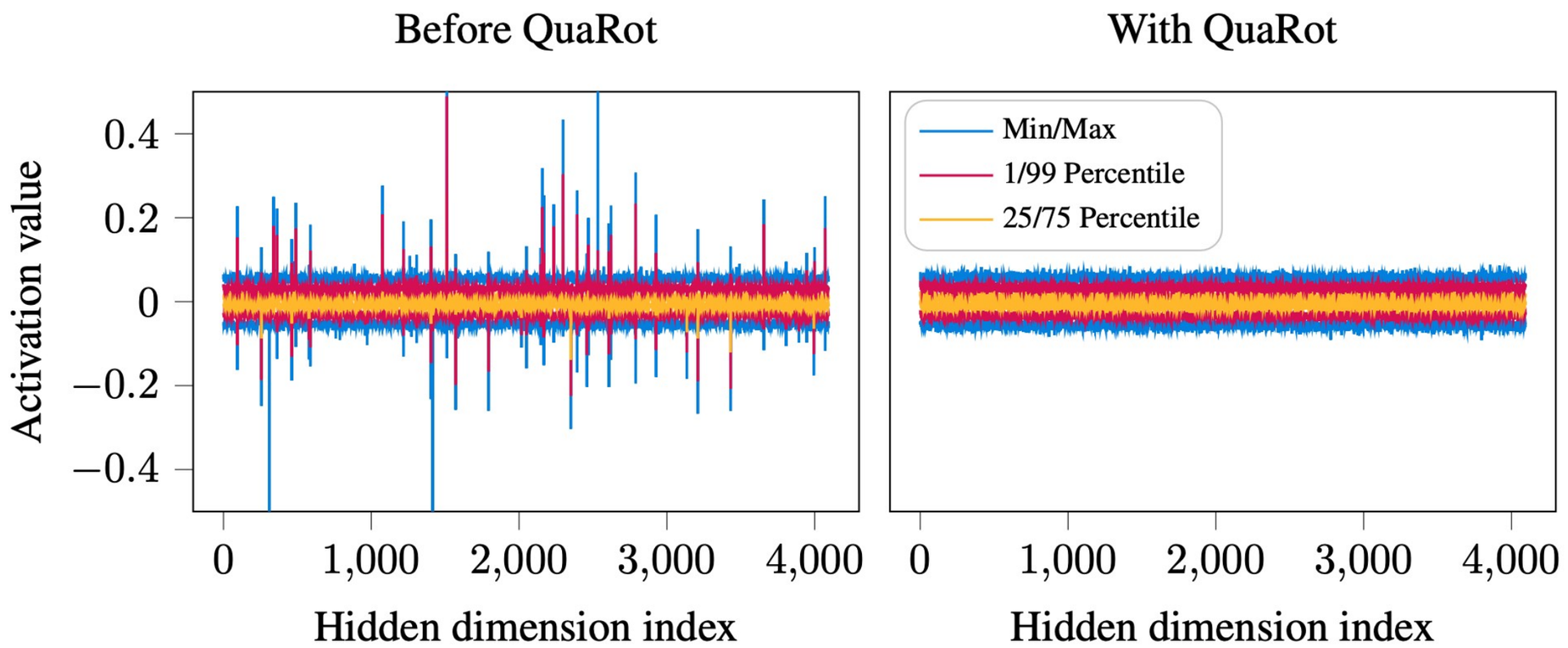
Closest prior art: where the contribution should lie

Not a new format: focus on AutoRound optimization, XPU deployment, and MoE integration.

Direction	Closest prior art	Proposed contribution
Channel reordering	RPTQ； DuQuant	Reconstruction-driven, locality-constrained search
MXFP4 + rotations	DuQuant++； FP-Quant	Joint selection with AutoRound rounding
High-precision subset	LLM.int8 / QUIK / Atom	W4 weights by default; A4/A8 activations
Mixed-MX channels	MicroMix (ICLR 2026 branch)	XPU split ABI; MoE sharing and bucketing
Data-aware transforms	SpinQuant； WUSH	Cost-matched advanced research; not MVP

QuaRot: paired rotations, not one-sided outlier handling

Original author figure, integer INT4 work. Illustrates rotations; not a new MXFP4 or XPU result.



Preserve the function; optimize the quantization error.

Source R05 · spcl/QuaRot@5008669 · Apache-2.0

DuQuant: rotate → permute → rotate

Author figure: handle within-block outliers, then balance blocks. Random shuffling is not guaranteed to help.

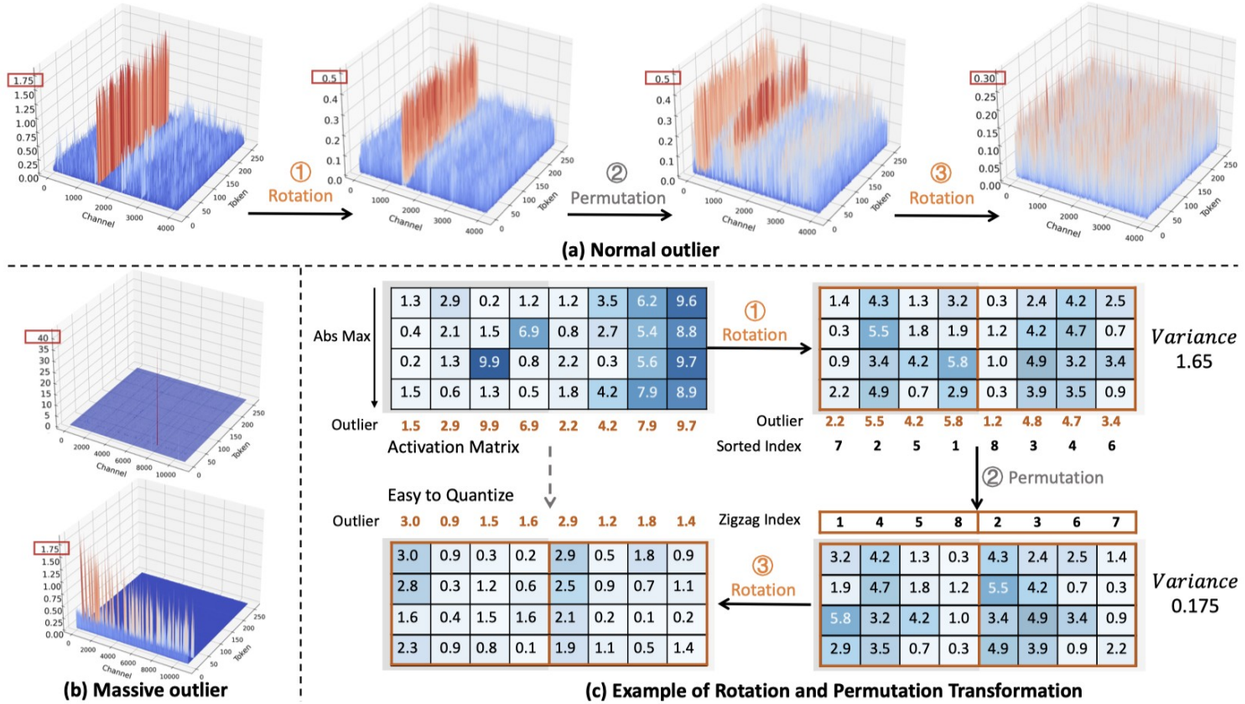


Figure: Transformation Steps for Activation Matrices after smooth technique. (a) Sequential transformations on Normal Outliers: ① initial rotation to reduce outliers within blocks, ② permutation to evenly distribute outliers across blocks, and ③ a second rotation for further smoothing. (b) Activation changes for Massive Outliers before and after DuQuant. (c) A sample matrix for highlighting the continual reduction of outliers through rotation and permutation, with outliers marked in dark blue.

Source R03 · MIT. R04 is the separate, later MXFP4 implementation.

Two grouping objectives

Without HMT: cluster magnitudes

With HMT: test outlier balancing

Validate both with complete QDQ

A direct MXFP4 baseline

DuQuant++ (2026)

Fine-grained, dimension-32 rotation

Align its extra scale conventions

DuQuant results: preserve the format and context

Integer W4A4; 1-* / 2-* denote LLaMA-1 / LLaMA-2. Lower perplexity is better.

Dataset	#Bit	Method	1-7B	1-13B	1-30B	1-65B	2-7B	2-13B	2-70B
WikiText2	FP16	-	5.68	5.09	4.10	3.53	5.47	4.88	3.31
	W4A4	SmoothQuant	25.25	40.05	192.40	275.53	83.12	35.88	26.01
		OmniQuant	11.26	10.87	10.33	9.17	14.26	12.30	NaN
		AffineQuant	10.28	10.32	9.35	-	12.69	11.45	-
		QLLM	9.65	8.41	8.37	6.87	11.75	9.09	7.00
		Atom	8.15	7.43	6.52	5.14	8.40	6.96	NaN
		DuQuant	6.40	5.65	4.72	4.13	6.28	5.42	3.79
		DuQuant+LWC	**6.18**	**5.47**	**4.55**	**3.93**	**6.08**	**5.33**	**3.76**
C4	FP16		7.08	6.61	5.98	5.62	6.97	6.46	5.52
	W4A4	SmoothQuant	32.32	47.18	122.38	244.35	77.27	43.19	34.61
		OmniQuant	14.51	13.78	12.49	11.28	18.02	14.55	NaN
		AffineQuant	13.64	13.44	11.58	-	15.76	13.97	-
		QLLM	12.29	10.58	11.51	8.98	13.26	11.13	8.89
		Atom	10.34	9.57	8.56	8.17	10.96	9.12	NaN
		DuQuant	7.84	7.16	6.45	6.03	7.90	7.05	5.87
		DuQuant+LWC	**7.73**	**7.07**	**6.37**	**5.93**	**7.79**	**7.02**	**5.85**

Source R03 · Hsu1023/DuQuant@d56cfc6 · MIT

How to read

Compare like for like.

Keep NaN and gaps.

INT4 is not MXFP4.

No Intel speed claim.

MicroMix: the closest implementation precedent for F2

Original concept diagram; no copied artwork or code. Inspected the pinned micromix branch.

Channel statistics

Collect activation statistics

Produce a fixed ordering

Ratios and ordering

Contiguous precision regions

Original: 4 / 6 / 8

Mixed GEMM

W4A4 / W4A8 and more

Blackwell kernels

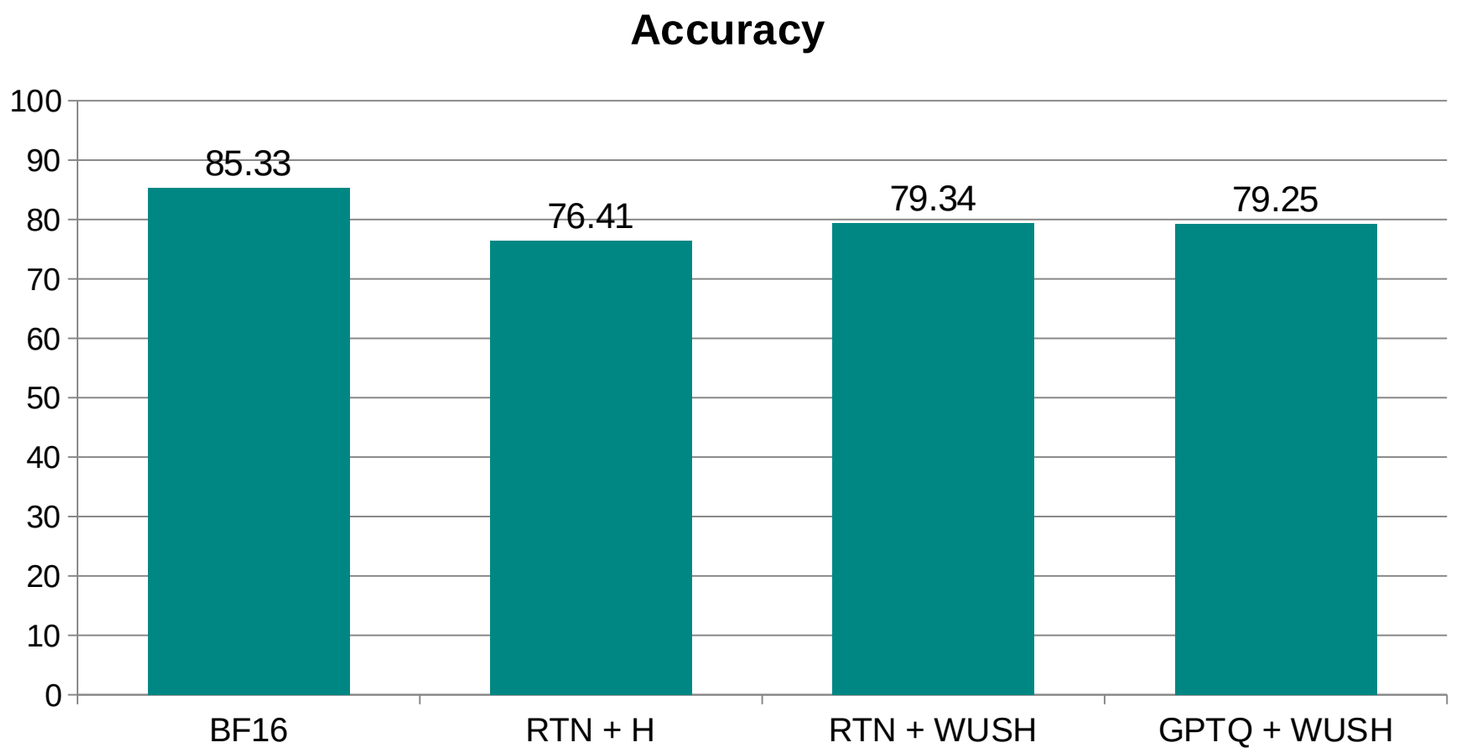
Verified MXFP8 activations × MXFP4 weights. Keep the high branch W4A8 by default.

Its 128-channel alignment is implementation-specific; determine XPU constraints separately.

MVP: two A4/A8 partitions. Defer FP6 numeric and deployment complexity.

MXFP4 evidence: fixed HMT leaves room to improve

WUSH author data · Llama-3.1-8B-Instruct · Macro accuracy over 13 Platinum tasks (%)



Author method	Accuracy	Count
BF16	85.33	1
RTN-H	76.41	5
RTN-WUSH	79.34	5
GPTQ-WUSH	79.25	5

+2.93 percentage points

RTN-WUSH versus RTN-H

Still 5.99 pp below BF16

Records are not verified seeds

Source R12 · Original CSV included · Editable PowerPoint chart

Keep MXFP4, MXFP8, and NVFP4 distinct

Align numeric profiles before comparing any reported numbers.

Format	Element	Shared scale	Role in this design
MXFP4	E2M1; maximum magnitude 6	E8M0, 32 elements	Low activations; all weights by default
MXFP8	E4M3 default; E5M2 optional	E8M0, 32 elements	High activation partition
NVFP4	E2M1	Different groups and scale conventions	Separate baseline; do not mix results

MXFP4 magnitudes: 0 · 0.5 · 1 · 1.5 · 2 · 3 · 4 · 6

ARK, core floor/rceil, and CUDA HMT use different numeric conventions.

Freeze rounding, zero blocks, signed zero, scales, and nibble order across calibration and export.

Invariant: transform activations and weight columns together

Convention: X[M,K], W[N,K], Y = X W^T + b.

X' = X[:, p] W' = W[:, p] X' (W')^T = X W^T

Orthogonal HMT

R R^T = I

X' = X P R

W' = W P R

General invertible T

X' = X T

W' = W T^(-T)

WUSH needs inverse-transpose compensation

Permute K channels, not tokens. The N-dimensional output needs no inverse permutation.

F1: optimize cross-block membership, not shuffling itself

Without changing quantization-block membership, reordering is a negative control, not a source of gain.

Original order (illustration)	0.4	7.8	0.3	6.2	0.5	9.1
Candidate order	0.3	0.4	0.5	6.2	7.8	9.1

Without HMT: cluster magnitudes

Sort by RMS, quantile, or log magnitude

Include weight norms and quantization residuals

With HMT: test outlier balancing

Zigzag, block swaps, locality constraints

Validate with complete W4A4 output loss

Original illustrative values and boxes; actual shared-scale groups contain 32 elements.

F1 search: use reconstruction loss, not amax alone

Cheap screening, then short AutoRound runs. Freeze permutations during tuning.

Candidate	Cost / objective	Selection rule
identity / random	Required baselines	Fixed seeds; cross-block and within-block controls
RMS / quantile sort	O(K log K)	Similar magnitudes; include weight sensitivity
zigzag + HMT	Balance outliers across rotation blocks	Compare directly against clustering
Local swaps	Bounded 128/256/512-channel windows	Accuracy gain + actual gather cost

$$\text{Loss} = \|Y_{ref} - Y_{qdq}\|^2 / (\|Y_{ref}\|^2 + \varepsilon)$$

Jointly normalize losses for shared-input consumers such as QKV and gate/up; reuse the gather.

Grouped GEMM: separate M-axis routing and K-axis gathers

These are different reorderings. Preserve top-k weights and the final scatter correspondence.

token × top-k

Original token ID

Router weight

Sort by expert

M-axis gather

Expert segments

Channel plan

K-axis gather

Shared / expert plan

grouped GEMM

Accumulate both partitions

scatter/reduce

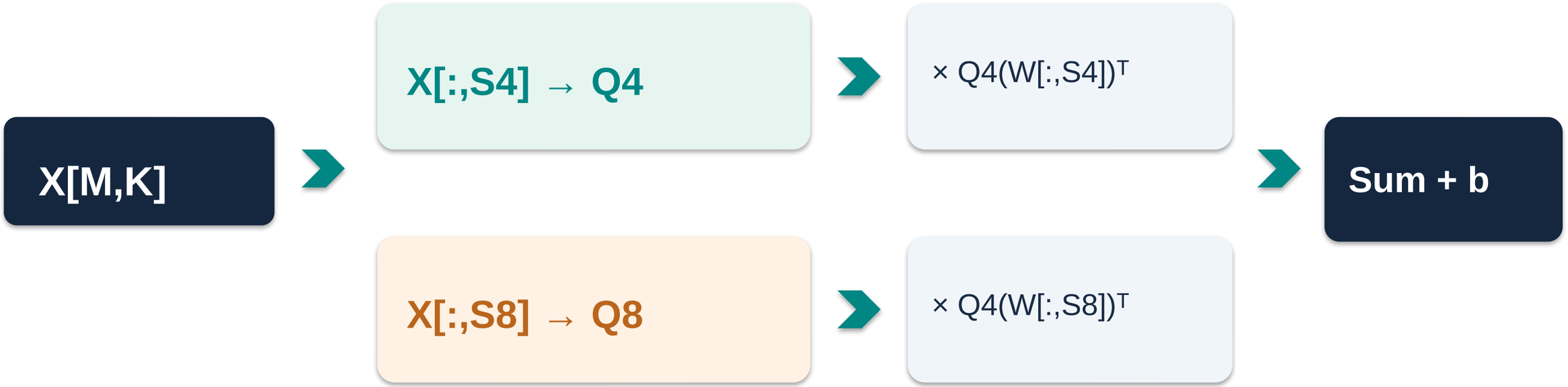
MVP: shared plans by expert group; gate/up share preprocessing.

Next: expert masks with fixed K8 buckets, grouped by shape, profile, and HMT.

Down-proj consumes expert-specific intermediate activations; gate/up plans cannot be reused unconditionally.

F2: one gather, two contiguous partitions, sum over K

Default: W4A4 low branch + W4A8 high branch, not two full-K GEMMs.



S4 and S8 are disjoint and cover all K. Add bias exactly once.

Report the three high-branch semantics separately

Stored format, compute operand representation, and quantization precision are different concepts.

Variant	High-partition weights	Interpretation
Default W4A8	Original weights → MXFP4	Only activation precision increases
W4 codes → W8 operands	Lossless E2M1 → E4M3 code expansion	Weight values unchanged; resident caches may grow
Optional W8A8	Original weights → MXFP8	Higher weight precision too; separate budget and ablation

Finite E2M1 values are exactly representable in E4M3; validate scale layout and device numerics.

If A8W4 is unsupported, explicitly select lossless expansion or decode.

Relabeling an INT4/INT8 kernel does not make it an MXFP4/MXFP8 kernel.

Who gets A8? Error reduction first, magnitude as a proxy

Moving channels changes entire shared-scale groups. Per-channel gains are not inherently additive.

Cheap candidate scores

RMS × weight-column norm

Upper quantile / tail frequency

A4→A8 quantization-error gain

Recompute QDQ

Regather and rebuild groups

Recompute all partition scales

Measure full-layer output error

Then allocate the budget

Swap boundary channels

Compare at matched actual K8

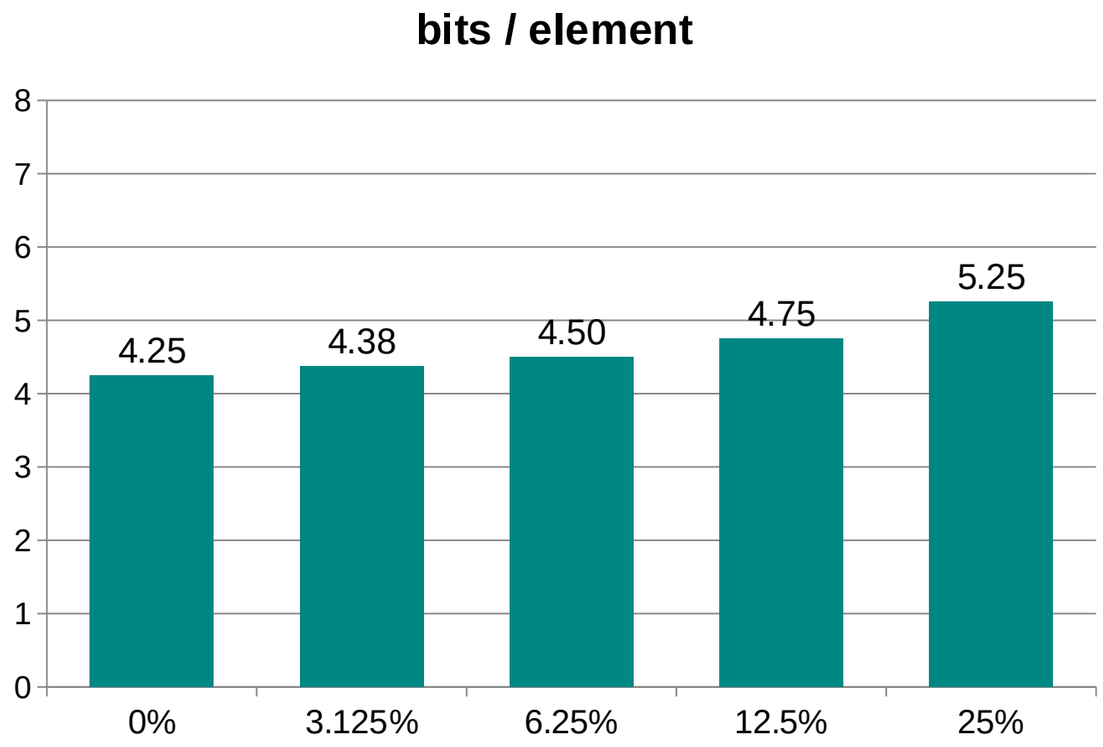
Allocate model-wide gain / cost

$$s[j] \approx ||Q4(W)[:,j]||^2 \times E[err4(xj)^2 - err8(xj)^2]$$

A screening proxy only: decide using complete post-partition QDQ.

Small ratios are discrete: alignment sets the actual budget

Theoretical activation payload only: excludes padding, indices, layouts, and expanded caches. Not a benchmark.



K	Target	K8	Actual
768	1%	32	4.17%
2048	1%	32	1.56%
4096	1%	64	1.56%
4096	5%	224	5.47%

4.25 + 4ρ bits / activation element

6.25% A8 → 4.50 bits; payload +5.88%.

No default increase in checkpoint weight bits.

HMT combinations: define the mask coordinate basis

Start with H1. In general, PR and RP are not equal.

Variant	Order	Purpose / risk
H0	Partition → A4/A8; no HMT	Isolate F1/F2 effects
H1 / recommended	Select in original basis → partition → low-only HMT	Keep sensitive channels out of low precision
H2	Full-input HMT → recalibrate channel selection	Old-basis masks are invalid
H3	Partition → independent HMT in each branch	Padding / overhead at small high-partition K
H4	Rotate → zigzag → rotate	DuQuant-style; two transforms are not free

Compare H32 and random-sign H32 first, then 128/256/512. MX quantization groups stay at 32.

Unified search: alternate discrete plans and AutoRound

Restore or rebuild tuning state: do not reuse gradients, scales, or statistics from the old ordering.

01

Stats / candidates

Route-weighted statistics

identity / cluster / zigzag

02

Partitions / HMT

Use the correct S8 basis

Recompute scales + QDQ

03

Short tuning

AutoRound: top 2–4 plans

Rerank on selection data

04

Cost selection

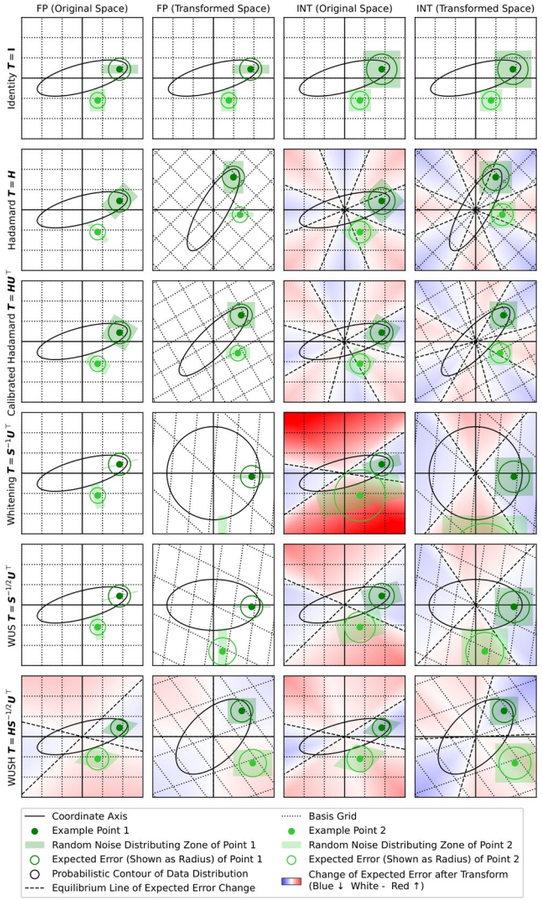
Measured XPU-cost Pareto

Freeze plan; final tuning

$$J = \text{reconstruction loss} + \lambda \times \text{measured total latency} + \mu \times \text{resident bytes}$$

Beyond fixed HMT: data-aware transforms

Original WUSH figure: not every optimal candidate is an orthogonal rotation.



Why compare?

Use weight and activation second moments

Fixed Hadamard need not be optimal

Theory assumptions are not task guarantees

Why not the MVP default?

Non-orthogonal T needs W T^(-T)

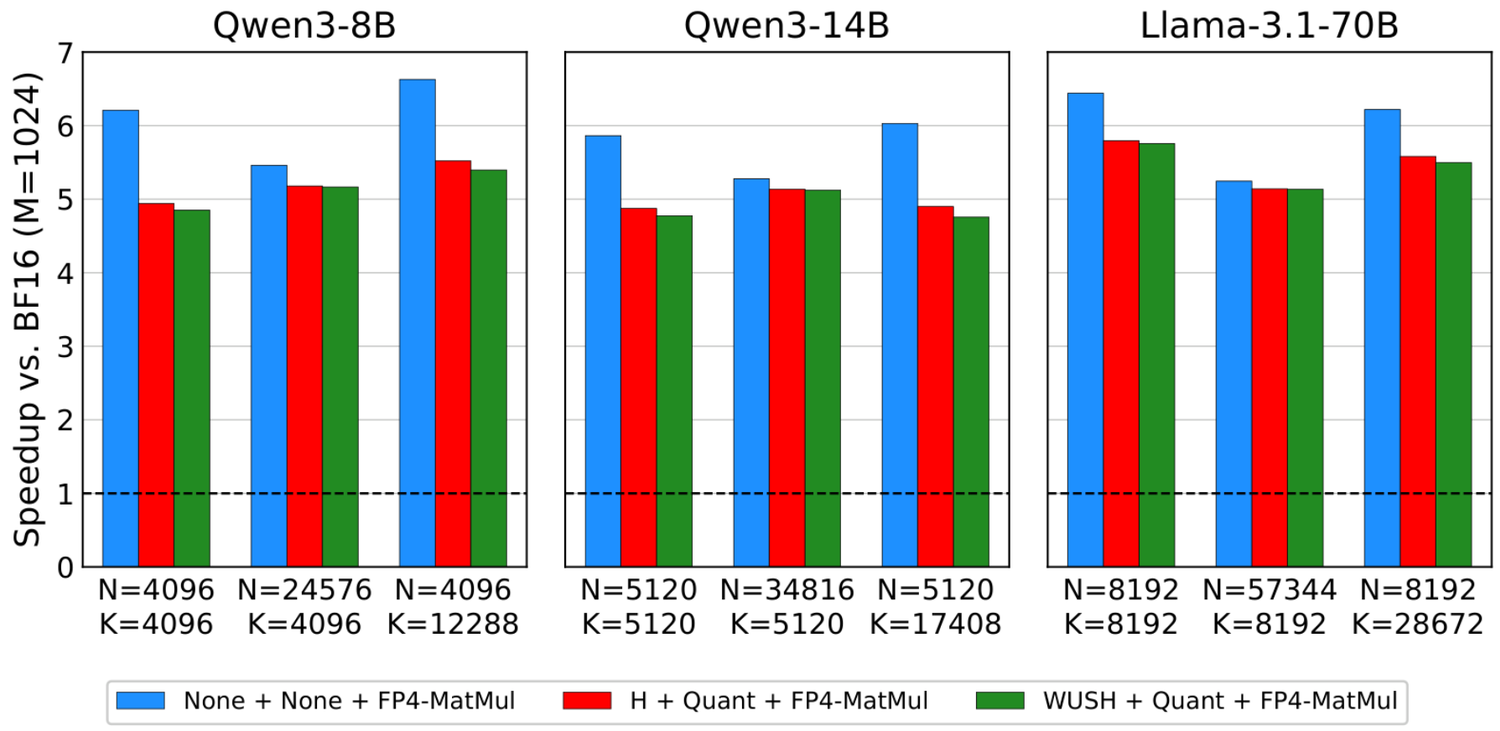
Measure dense-block and scale overhead

Compare learned rotations at matched cost

Source R12 · Apache-2.0 · Zoom the original figure; commentary is original to this report.

What a paper speed chart can and cannot establish

WUSH author figure 2: per-layer speedup vs. BF16 at M=1024. Not end-to-end or Intel data.



Include transform and quantization overhead; some kernel gains may still remain.

The figure/script does not establish a GPU model. No cross-hardware claim. Source R12 · Apache-2.0.

AutoRound integration: extend existing mechanisms

Baseline 5db1082. A preprocessing / quantization plan, not a new scalar datatype.

Layer	Existing mechanism	Required extension
Schemes / numerics	schemes.py； data_type/mxfp.py	Explicit plan schema; reuse 4/8-bit QDQ
Composition / calibration	composer； BasePreprocessor	Freeze plans; invalidate references; basis-aware stats
Tuning / grouped	wrapper； grouped_experts.py	Partition QDQ, gradients, signatures, buckets
XPU kernels	ARK HMT + MXFP4 primitive	Gather / signs / split outputs + GEMM consumers
Persistence / inference	qlinear_fp； formats； backend	Two packed partitions; versioned reload

Default off. Preserve existing models, presets, and exported results.

Three real gaps: hooks, packed consumers, capabilities

Code existence does not establish integration into a model's actual inference path.

Grouped hooks

Online transform hooks disqualify some grouped fast paths.

Add explicit plan execution, not just hooks.

An existing HMT primitive

The XPU primitive emits packed FP4.

That does not establish dense / MoE

MXFP4 activation GEMM integration.

Existing W4A8 is not MX

ARK uses INT4 storage with INT8 DPAS compute.

Some FP8 paths await hardware validation.

Benchmark the actual dispatch, fallback reason, workspace, and code revision.

Intel XPU: probe native compute, do not assume it

Pinned oneDNN main commit 4149450; not a guarantee about the installed release.

Architecture	FP8	FP4 E2M1
Xe-HPC	Conversion	Conversion
Xe2-HPG	Conversion	Conversion
Xe3-LPG	Conversion	Conversion
Xe3p-LPG	Native	Conversion
Xe3p-XPC	Native	Native

Still probe the primitive: A8W4? Block-scale layouts? Grouped / ragged support?

Record GPU ID, driver, oneAPI, oneDNN, and ARK revisions. Explicitly reject unimplemented paths.

Kernel tiers: reference, decode-fused, verified native

One packed ABI and one numeric contract across three execution tiers.

Tier	Implementation	Benefits / limits
Reference	FP32 transform + QDQ; concatenated high-precision GEMM	Correctness oracle; its runtime is not low-bit kernel speed
Conversion devices	Decode FP4/FP8 on load; group scales; FP16/BF16 DPAS	Reduce storage / decode workspace; no native FP4 throughput claim
Native devices	Supported block-scaled ops; dual branches or fused accumulation	Verify every operand / shape / numeric profile

Fusion target: routing + gather + sign + HMT + quantize + pack

Persist one versioned plan across the entire lifecycle

Proposed schema, not a callable API. Do not pass unknown fields to a filtering from_dict.

Metadata	Tensor buffers	Load-time validation
version / profile / transform_order	Actual permutation and signs	Dimensions, valid permutation, hashes
mask_basis / logical K / padded K4,K8	Partitioned packed weights + E8M0 scales	Disjoint, full K coverage, group boundaries
Shared-plan links / transformed-weight flag	Optional expanded caches are not original weights	Apply once; prevent double rotation

Keep the same plan through save → shard/offload → load → generate.

safetensors buffers; legacy checkpoints; rank-local K permutations for initial TP support.

Reject unsupported backends; reference fallback must be explicitly enabled and logged.

Correctness: algebra first, then encoding, then models

Accumulation orders may differ; packed codes must match the corresponding numeric reference.

Level	Required tests
Algebra	Paired permutation, orthogonal / inverse transpose, one-sided negative tests, normalization
Encoding	Ties, zero groups / signed zero, extremes / subnormals, NaN/Inf, reserved E8M0
Partitions	Empty / full S8, tails, invalid indices rejected, one bias, retained padded coordinates
grouped	Empty experts, hotspots, top-k scatter, shared / independent plans, signatures / fallback
Lifecycle	Gradients / STE, quantize-save-load, immediate shards, meta/offload, legacy models

Disabled features must preserve behavior. Tiny random-model checks are not real-model accuracy evidence.

Experiments: fair budgets, separate data, real MoE loads

Tiny → dense → MoE. Include prefill and decode, not only large-M GEMM.

Axis	Recommended matrix
Baselines	BF16; existing MXFP4 RTN / AutoRound; F1, F2, combined; each ±HMT
Budgets	Match actual K8, all-W4, calibration data, tuning iterations; separate optional W8
Shapes	M=1/8/32/128/1024; real projections, e.g. K=768/2048/4096/11008
MoE	E=8/64/128, top-k=2/8; balanced, long-tail, empty experts, hotspots
Generalization	Separate calibration / selection / final data; code/math, long context, OOD; ≥3 seeds

Track routing coverage. Forced-coverage samples must not silently replace real token weights.

Proposed gates: development targets, not achieved results

Accuracy / compression research can succeed without low-bit arithmetic acceleration.

Quality gate / proposed

2 dense + 1 MoE model

Median selection-layer error down ≥20%

No material final PPL/task regression

At least one real improvement

Engineering gate / proposed

Start with K8≤6.25% buckets

Complete projection latency ≤1.10× baseline

Record peak / resident memory

Decide on full-model Pareto results

Total latency = routing + gather + HMT + quantization + both GEMMs + sum + scatter

Same device and budget; post-warmup median/p95. Report kernel / block / TTFT / TPOT separately.

Research priorities: four falsifiable questions first

Constrain later learned rotations, dynamic masks, and MXFP6 by actual implementation cost.

Priority	Direction	Falsifiable question
P0	Error-gain A8 selection	Better than maximum-magnitude selection at matched K8?
P0	HMT + A8	Complementary or substitutes? Can HMT reduce the needed A8 budget?
P0	Locality-constrained permutations	Retain accuracy while reducing random-gather cost?
P0	Shared / expert MoE plans	Do expert-specific gains justify top-k preprocessing duplication?
P1	Exponent boundaries, global budget, WUSH / SpinQuant	Better at matched cost? Is the correct compensation deployable?
P2	Dynamic masks, W8 / low-rank paths, MXFP6	Can additional accuracy and additional cost be attributed separately?

Milestones: numerics, dense, grouped, persistence

Person-days are effort estimates, not schedule commitments. Algorithm and kernel work can partly overlap.

Stage	Deliverable	Dependencies	Effort
M0	Device capabilities / numeric profile / packed ABI / baseline	Actual XPU	2–3 person-days
M1	CPU reference / F1-F2 dense / negative controls	M0	4–6
M2	HMT / search / AutoRound wrapper	M1	5–8
M3	Fused XPU preprocessing / dual-branch dense	M0/M1; HMT needs M2	6–10
M4	Shared expert plans / bucketing / decode	M2/M3	6–10
M5	Export-reload / shards-TP / full evaluation	M2/M3; MoE needs M4	5–8

Final decision: Pareto evidence, not isolated claims

More accurate, smaller, and faster are three different conclusions.

Complete M0 / M1 before selecting the kernel investment.

Keep static plans with real accuracy gains at a matched budget.

Advance XPU paths with controlled total cost and export/reload support.

Reject candidates that overfit calibration data or have excessive gather cost.

Core contribution: AutoRound × channel budgets × HMT × Intel XPU / MoE

References 1/2: papers and pinned implementations

Full titles, URLs, file SHA256, licenses, and evidence notes: evidence.json and speaker notes.

ID	Work / reference	Paper ID	Pinned commit
R01	Microscaling / OCP MX	2310.10537	7bc4195
R02	RPTQ	2304.01089	9619069
R03	DuQuant	2406.01721	d56cfc6
R04	DuQuant++	2604.17789	6c13ec0
R05	QuaRot	2404.00456	5008669
R06	SpinQuant	2405.16406	8f47aa3
R07	LLM.int8() / bitsandbytes	2208.07339	a616886

Only licensed MIT / Apache author assets are reproduced. No MicroMix / Atom / SpinQuant artwork.

References 2/2: papers and pinned implementations

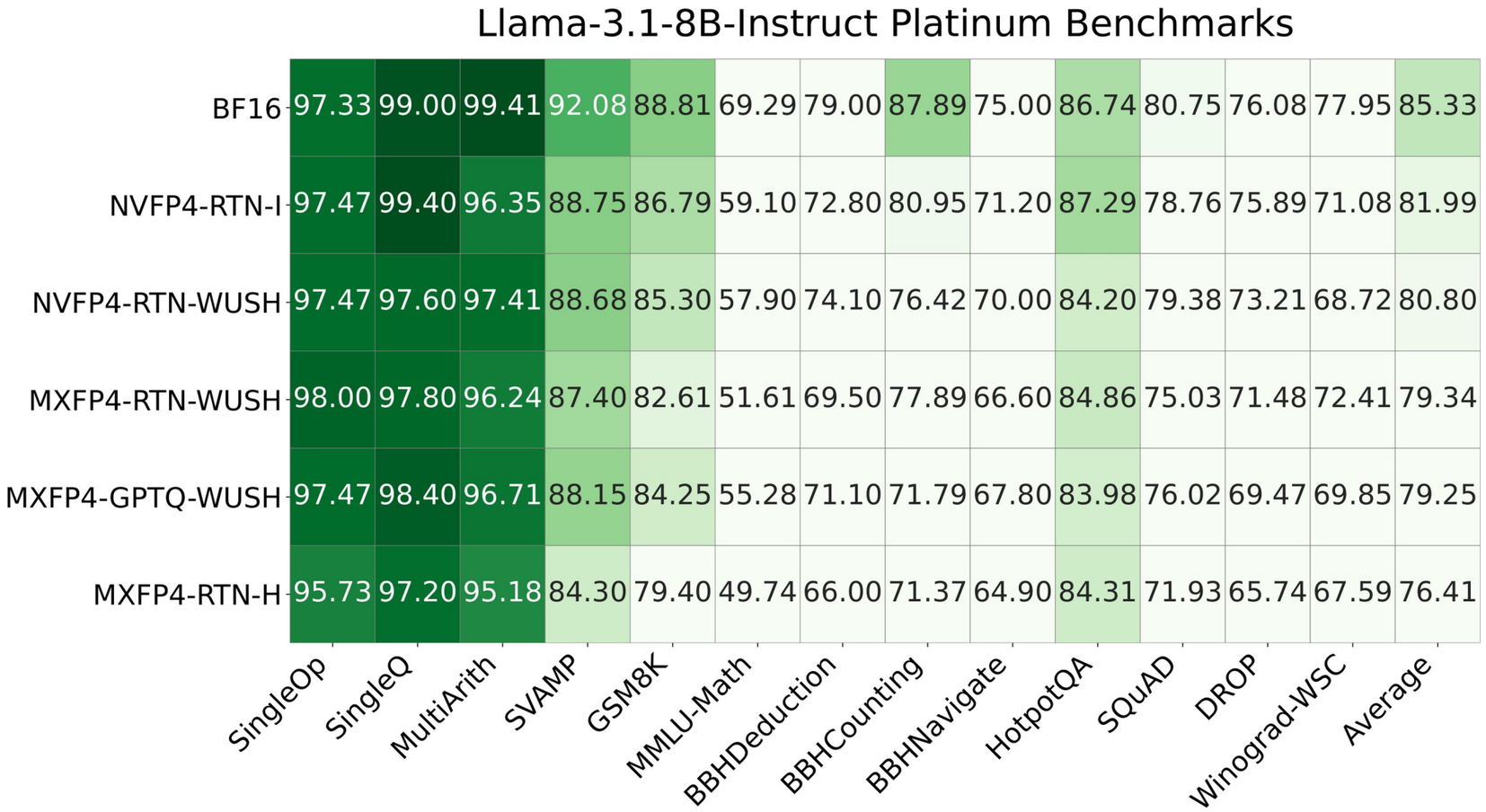
Full titles, URLs, file SHA256, licenses, and evidence notes: evidence.json and speaker notes.

ID	Work / reference	Paper ID	Pinned commit
R08	QUIK	2310.09259	9558d71
R09	Atom	2310.19102	7e3618b
R10	MicroMix	2508.02343	c57370b
R11	FP-Quant / MR-GPTQ	2509.23202	d2e3092
R12	WUSH	2512.00956	0b69f66
R13	oneDNN datatype / MXFP8 tutorial	Main-branch docs	4149450

Only licensed MIT / Apache author assets are reproduced. No MicroMix / Atom / SpinQuant artwork.

Appendix: the original WUSH task-level result table

Llama-3.1-8B-Instruct · Author-released figure 10 · Unmodified data and labels · Zoom to read



Source R12 · Apache-2.0 · Complete licenses in these notes and attached to the PDF.